

S P 500 price prediction

Written by Jeungsoo Noh

¹Northeastern University
360 Huntington ave, Boston, MA 02115 USA

Abstract

I investigated whether technical-indicator-based machine-learning models can meaningfully forecast one-day-ahead S&P 500 returns. To do so, I got the SP data from Nasdaq. My engineered indicators were (SMA, EMA, MACD, RSI, Bollinger bands), and the models tested were linear regression, Random Forest, XGBoost, SVR. and the evaluation metrics were MAE, MSE, RMSE, R2. Linear regression had the most reasonable results among the models; however, it was not accurate because of a random walk.

Introduction

This paper asks a single practical question. Do popular price-based indicators help common ML regressors forecast S&P 500 closing price next day? To answer it, I constructed the following.

- (i) sources daily S&P 500 data from NASDAQ (2020 – 2025)
- (ii) engineers five canonical indicators: SMA, EMA, MACD, RSI, and Bollinger Bands.
- (iii) evaluates four existing models of increasing complexity: ordinary least-squares (OLS) linear regression, support vector regression, random forest, and XGBoost.

All models are tested and used MAE, MSE, RMSE, and R2 for evaluation.

Background

The weak-form Efficient Market Hypothesis argues that prices already reflect all publicly available information, so tomorrow's return should be statistically indistinguishable from a random draw conditioned on today. Any model that uses only past prices must therefore outperform a "today = yesterday" benchmark to claim genuine predictive power. In this case, it failed to do so.

Technical Indicators

Traders use deterministic functions of historical price and volume—*technical indicators*—to infer momentum, mean-reversion, and changing volatility. The five most common are:

1. **Simple Moving Average (SMA)** – plain average over a fixed window.

2. **Exponential Moving Average (EMA)** – moving average with exponentially decreasing weights. This puts more weight on recent price.
3. **Moving-Average Convergence Divergence (MACD)** – difference between a fast and a slow EMA. Typically EMA 26 minus EMA 12
4. **Signal Line** – smoothed MACD line, providing a less volatile indication of the overall trend. This uses EMA 9.
5. **Relative Strength Index (RSI)** – bounded momentum oscillator comparing recent gains to losses.
6. **Bollinger Bands** – upper and lower bands at k standard deviations around a moving average, used as a volatility gauge.

Because these indicators are straightforward functions of price, they are easy inputs for both linear and non-linear machine-learning models.

Machine-Learning Regressors in Finance

Earlier research has applied a spectrum of algorithms to indicator data.

- **Linear models** (ordinary least squares, ridge) provide transparency but assume additive effects.
- **Kernel methods** such as Support-Vector Regression capture nonlinearities but are sensitive to hyperparameter choices. Perhaps this led to the failure to capture the true stock price.
- **Tree ensembles**—Random Forest and gradient-boosting implementations like XGBoost—automatically discover interactions and have become standard baselines in financial ML studies.

Despite sporadic success, published results remain mixed; many claims rely on fixed test sets rather than realistic, time-ordered evaluation.

Evaluation Protocols for Time-Series Forecasting

Financial series are non-stationary. A *walk-forward* or expanding-window evaluation, in which models are re-trained as new data arrive and predictions are made strictly one day ahead, mimics real trading far better than random splits. We report four standard regression metrics—MAE, MSE, RMSE, and R2 to remain comparable with prior work. Later on, ADF test was conducted to check whether data is stationary or not.

	Close	Open	High	Low	SMA_5	SMA_10	SMA_50	SMA_100	EMA_12
Date									
2020-08-25	3443.62	3435.95	3444.21	3425.84	3406.484	3393.082	3236.3812	3075.2085	3384.577679
2020-08-26	3478.73	3449.97	3481.07	3444.15	3427.260	3402.920	3243.4610	3085.1093	3399.062652
2020-08-27	3484.55	3485.14	3501.38	3468.35	3447.068	3414.032	3250.8822	3093.3180	3412.214551
2020-08-28	3508.01	3494.69	3509.23	3484.32	3469.238	3427.548	3258.7356	3101.8040	3426.952313
2020-08-31	3500.31	3509.73	3514.77	3493.25	3483.044	3439.380	3266.7870	3109.3073	3438.238111

Figure 1: Data Preprocessing

```
def calculate_sma(prices, period):
    sma_values = []
    for i in range(len(prices)):
        if i < period - 1:
            sma_values.append(None)
        else:
            sma = np.mean(prices[i-period + 1 : i + 1])
            sma_values.append(sma)
    return sma_values

df["SMA_5"] = df["Close"].rolling(window=5, min_periods=5).mean()
df["SMA_10"] = df["Close"].rolling(window=10, min_periods=10).mean()
# Long term
df["SMA_50"] = df["Close"].rolling(window=50, min_periods=50).mean()
df["SMA_100"] = df["Close"].rolling(window=100, min_periods=100).mean()
df["SMA_100"].head(101)
```

Figure 2: SMA calculation

These concepts—market efficiency, indicator design, choice of ML regressors, and proper evaluation—frame the methodological context for our study of next-day S&P 500 return prediction.

Project Description

5.1 Data

- **Source & horizon.** Daily S&P 500 index “Adj Close” prices retrieved from NASDAQ Data 3 April 2020 – 2 April 2025.
- **Data Preprocessing** Using SimpleImputer resulted in duplicating the values. SMA 100 had 99 duplicate values due to filling in.

As shown in figure 1, I chose to start calculating when all the calculations appear, which is 08-25-2020

5.2 Feature Engineering SMA: Simple Moving Average is displayed in Figure 2

EMA: Exponential Moving Average is displayed in Figure 3

From EMA, MACD(Moving Average Convergence/Divergence, EMA26 minus EMA 12) and Signal line(EMA 9) can be derived. It is displayed in figure 4

The histogram can be drawn from this and will appear at visualization section

- **RSI : Relative Strength Index**, Bollinger band are calculated as following:

I used Standard deviation of 2, which is the standard way to calculate RSI.

```
def calculate_ema(prices, period):
    ema_values = []
    alpha = 2 / (period + 1)
    ema_prev = None

    for i in range(len(prices)):
        price = prices[i]

        # Skip if price is None or NaN
        if price is None or np.isnan(price):
            ema_values.append(None)
            continue

        if i < period - 1:
            ema_values.append(None)
        elif i == period - 1:
            valid_prices = [p for p in prices[:period] if p is not None and not np.isnan(p)]
            sma = np.mean(valid_prices)
            ema_values.append(sma)
            ema_prev = sma
        else:
            if ema_prev is None or np.isnan(ema_prev):
                ema_values.append(None)
                continue
            ema = alpha * price + (1 - alpha) * ema_prev
            ema_values.append(ema)
            ema_prev = ema

    return ema_values
```

Figure 3: EMA calculation

```
df['EMA_12'] = calculate_ema(df['Close'].tolist(), 12)
df['EMA_26'] = calculate_ema(df['Close'].tolist(), 26)

# Drop rows where EMA_12 or EMA_26 is not available
df.dropna(subset=['EMA_12', 'EMA_26'], inplace=True)

df['MACD_Line'] = df['EMA_12'] - df['EMA_26']
df['Signal_Line'] = calculate_ema(df['MACD_Line'].tolist(), 9)

df['MACD_Histogram'] = df['MACD_Line'] - df['Signal_Line']
```

Figure 4: MACD and Signal Line

```
def calculate_rsi(prices, period=14):
    prices = pd.to_numeric(prices, errors='coerce').ffill()
    delta = prices.diff()
    gain = np.where(delta > 0, delta, 0)
    loss = np.where(delta < 0, -delta, 0)

    # Calculate rolling mean for gains and losses
    avg_gain = pd.Series(gain, index=prices.index).rolling(window=period).mean()
    avg_loss = pd.Series(loss, index=prices.index).rolling(window=period).mean()

    # Prevent division by zero
    rs = np.where(avg_loss == 0, 100, avg_gain / (avg_loss + 1e-10))
    rsi = 100 - (100 / (1 + rs))

    return pd.Series(rsi, index=prices.index)
# RSI is zero if average gain is zero

# Bollinger Bands Calculation Function
def calculate_bollinger_bands(prices, window=20, num_std=2):
    sma = prices.rolling(window=window).mean()
    std = prices.rolling(window=window).std()

    # STD = 2 SD (Default): Works well in most market conditions.
    # STD = 3 : Ideal for highly volatile markets or large-cap stocks where volatility is more extreme
    upper_band = sma + (num_std * std) # Upper = SMA 20 + 2 std
    lower_band = sma - (num_std * std) # Lower = SMA 20 - 2 std
    # Middle = SMA 20
    return sma, upper_band, lower_band
# Lower std will have less accuracy.

close_prices = df["Close"]

# Calculate RSI
df["RSI_14"] = calculate_rsi(close_prices)

# Calculate Bollinger Bands
df["BB_Middle"], df["BB_Upper"], df["BB_Lower"] = calculate_bollinger_bands(close_prices)
```

Figure 5: RSI and Bollinger band



Visualization of Features

Figure 6 shows result of feature engineering after data preprocessing

Using simpleImputer resulted in abnormal line among features. Therefore, the magnitude of missing value was resolved by calculating all features based on the longest calculation to be executed, which is SMA 100. Brief Explanation:

- EMA tells how recent price is moving.
- MACD line & signal line = Buy signal, and vice versa for other way around.
- Bollinger Band tells how volatile the price is.

Algorithm used

There are total of four algorithms used for this project:

Linear Regression

•Random Forest

•XGBoost

•Support Vector Regression(SVR)

Model implementation and evaluation

The goal is to test whether these models can predict the next day's closing price. MSE, RMSE, MAE, R2 were used to evaluate the model. The actual closing price of April 3rd was 5492.74

Figure 6 is a code to train the model.

Linear Regression

Linear Regression(Figure 7) achieved an **R² score of 0.965**, meaning it could explain over 96% of the variance in the next-day price. The **MAE** was around 39, which is quite reasonable for financial data. Predicted price for next day is far off from actual(5492.74). However, it is still the closest estimator.

Random Forest It severely underfit the data(Figure 8). In contrast to linear regression, MSE is 326398, and had negative R2 score.

XGBoost

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False # crucial for time series
)
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Figure 6: Model Training

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error as MSE
from sklearn.metrics import mean_absolute_error as MAE
from sklearn.metrics import r2_score
lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)
y_pred = lr_model.predict(X_test_scaled)

mse = MSE(y_test, y_pred)
rmse = np.sqrt(MSE(y_test, y_pred))
mae = MAE(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R2 Score:", r2)

MSE: 2633.1036909073173
RMSE: 51.31377681390561
MAE: 39.12822856911246
R2 Score: 0.9648015936268225

X_test_scaled[-1] # Next day
lr_model.predict([X_test_scaled[-1]]) # prediction

array([5624.54732478])
```

Figure 7: Linear Regression

```
#Random Forest
from sklearn.ensemble import RandomForestRegressor
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train_scaled, y_train)
mse = MSE(y_test, y_pred_RF)
rmse = np.sqrt(MSE(y_test, y_pred_RF))
mae = MAE(y_test, y_pred_RF)
r2 = r2_score(y_test, y_pred_RF)

print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R2 Score:", r2)

MSE: 326398.36325486185
RMSE: 571.31284184312
MAE: 506.0950306034461
R2 Score: -3.363178810359341

rf_model.predict([X_test_scaled[-1]])

array([5186.1841])
```

Figure 8: Random Forest

```
#XG BOOST
!pip install xgboost
from xgboost import XGBRegressor
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
xgb_model = XGBRegressor(n_estimators=100, random_state=42)
xgb_model.fit(X_train_scaled, y_train)
y_pred_xg = xgb_model.predict(X_test)

mse = mean_squared_error(y_test, y_pred_xg)
rmse = mean_squared_error(y_test, y_pred_xg)
mae = MAE(y_test, y_pred_xg)
r2 = r2_score(y_test, y_pred_xg)

print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R2 Score:", r2)
```

Requirement already satisfied: xgboost in d:\anaconda\lib\site-packages (3.0.0)
Requirement already satisfied: numpy in d:\anaconda\lib\site-packages (from xgboost) (1.26.4)
Requirement already satisfied: scipy in d:\anaconda\lib\site-packages (from xgboost) (1.13.1)
MSE: 456827.91703448223
RMSE: 456827.91703448223
MAE: 617.8282032933728
R2 Score: -5.106715326967125

```
xgb_model.predict([X_test_scaled[-1]])
```

array([5123.9004], dtype=float32)

Figure 9: XGBoost

Likewise, the MSE, and MAE are substantially higher than linear regression. XGboost also underfits the data. Grid search was used later again for this process. Result is following: All four evaluating factors decreased but it still did not bring huge difference to results. Prediction is far off from actual(5492.74)

SVR(Support Vector Regression)

SVR, even after scaling, still failed to generalize, possibly due to overfitting or sensitivity to hyperparameters. I manually set the C, gamma and epsilon for SVR. Even changing these inputs, SVR failed the worst. It had R2 score of -29, MAE is the highest among other models. Prediction, like other model, is far off from actual(5492.74)

Result Interpretation and Future Direction

Result

Surprisingly, even simple model like Linear Regression can somewhat perform well. All models treated the input features like **static data**, not time-series. I used time series data but did not conduct time series models like LSTM. Models' prediction is following: "Given today's indicators, predict tomorrow's price. Not using lag/sequence models and taking past values as features (e.g., $Close(t-1)$, $Close(t-2)$), along with no autocorrelation modeling could have led to the failure of the models used above.

The Efficient Market Hypothesis, it still will be hard to predict the stock price. Stock prices reflect all information, public and private. It is a Stochastic Process, or random walk. That is, stockholder's reaction to the market is unable

```
#XG BOOST
!pip install xgboost
from xgboost import XGBRegressor
from sklearn.model_selection import GridSearchCV
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [3, 5],
    'learning_rate': [0.01, 0.1],
    'subsample': [0.7, 1.0],
    'colsample_bytree': [0.7, 1.0]
}
xgb_model = XGBRegressor(random_state=42, verbosity=0)
grid_search = GridSearchCV(xgb_model, param_grid, scoring='neg_mean_squared_error', cv=3, verbose=1, n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)
best_xgb = grid_search.best_estimator_
y_pred_xg = best_xgb.predict(X_test_scaled)

mse = MSE(y_test, y_pred_xg)
rmse = np.sqrt(MSE(y_test, y_pred_xg))
mae = MAE(y_test, y_pred_xg)
r2 = r2_score(y_test, y_pred_xg)
best_params = grid_search.best_params_
print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R2 Score:", r2)
```

Requirement already satisfied: xgboost in d:\anaconda\lib\site-packages (3.0.0)
Requirement already satisfied: numpy in d:\anaconda\lib\site-packages (from xgboost) (1.26.4)
Requirement already satisfied: scipy in d:\anaconda\lib\site-packages (from xgboost) (1.13.1)
Fitting 3 folds for each of 32 candidates, totalling 96 fits
MSE: 329903.68730057907
RMSE: 574.372429161236
MAE: 509.1455096646013
R2 Score: -3.410036753506649

```
xgb_model.predict([X_test_scaled[-1]])
```

array([5123.9004], dtype=float32)

Figure 10: GridsearchCV

to predict. There may be a model that roughly predict for certain amount of time, but it may not work due to random walk. LSTM models could have yielded different result.

Above is the Augmented Dickey Fuller Test to see whether data is stationary or not. As expected, the data is non-stationary.

Future direction

Coming up with the accurate machine learning model to predict stock price will remain challenging problem. The effect of noise in stock market is immense. The noise is directly related to the result of the market: tariff, changes, tax, interest rate, performance of companies, etc. Even if the model works well for today, the announcement by U.S President or CEO will make the model predict inaccurate results. Main issue that machine learning scientists to deal with when it comes to predicting stock price is random walk. Unfortunately, no one can predict tomorrow's announcement by president or any possible change due to stochastic process. I would have done LSTM or ARIMA to test whether there is a significant difference in terms of accuracy and prediction compared to four models I listed.

References

NASA. 2015. Pluto: The 'Other' Red Planet. <https://www.nasa.gov/nh/pluto-the-other-red-planet>. Accessed: 2018-12-06.

```

#SVR Support Vector Regression
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
from sklearn.svm import SVR
svr = SVR(kernel='rbf', C=100, gamma=0.1, epsilon=0.1)
svr.fit(X_train, y_train)
y_pred_svr = svr.predict(X_test)

# Evaluate
mse = MSE(y_test, y_pred_svr)
rmse = np.sqrt(mse)
mae = MAE(y_test, y_pred_svr)
r2 = r2_score(y_test, y_pred_svr)

print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R2 Score:", r2)

```

```

MSE: 2259664.5856648227
RMSE: 1503.2180765493817
MAE: 1478.1262181367774
R2 Score: -29.206403427929306

```

```
svr.predict([X_test_scaled[-1]])
```

```
array([5252.20649351])
```

Figure 11: SVR

```

from statsmodels.tsa.stattools import adfuller
# Perform the Augmented Dickey-Fuller (ADF) Test
adf_test = adfuller(df['Close'])
adf_results = {
    'ADF Statistic': adf_test[0],
    'p-value': adf_test[1],
    'Critical Values': adf_test[4]
}
adf_results
# result shows it is fail to reject. so data is non-stationary.

{'ADF Statistic': -1.0074797512010436,
 'p-value': 0.7505264670247752,
 'Critical Values': {'1%': -3.4360194465416387,
 '5%': -2.8640434537995523,
 '10%': -2.5681028978640104}}

```

Figure 12: Augmented Dickey-Fuller Test